Function-Oriented Software Design

- In this design technique, we view a system as a black-box that provides a set of services to the users of the software.
- These services provided by a software to its users are also known as the ***high-level functions*** supported by the software.
- During the design process, these high-level functions are successively decomposed into more detailed functions.

> The term ***top-down decomposition*** is often used to denote the successive decomposition of a set of high-level functions into more detailed functions.

- After top-down decomposition has been carried out, the different identified functions are mapped to modules and a module structure is created.

Overview of SA/SD Methodology

- SA/SD methodology involves carrying out two distinct activities:
  - **Structured Analysis (SA)**
  - **Structured Design (SD)**

Structured Analysis

Introduction

- During structured analysis, the SRS document is transformed into a data flow diagram (DFD) model.
- During structured analysis, functional decomposition of the system is achieved. Each function that the system needs to perform is analysed and hierarchically decomposed into more detailed functions.

Data Flow Diagrams (DFDs)

- A DFD is a hierarchical graphical model of a system that shows the different processing activities or functions that the system performs and the data interchange among those functions.
- DFD model only represents the data flow aspects it completely ignores aspects such as control flow, the specific algorithms used by the functions, etc.

- In the DFD terminology, each function is called a ***process*** or a ***bubble***.
- It is useful to consider each function as a processing station (or process) that consumes some input data and produces some output data.

Primitive symbols user for constructing DFDs

- There are essentially five different types of symbols used for constructing DFDs.

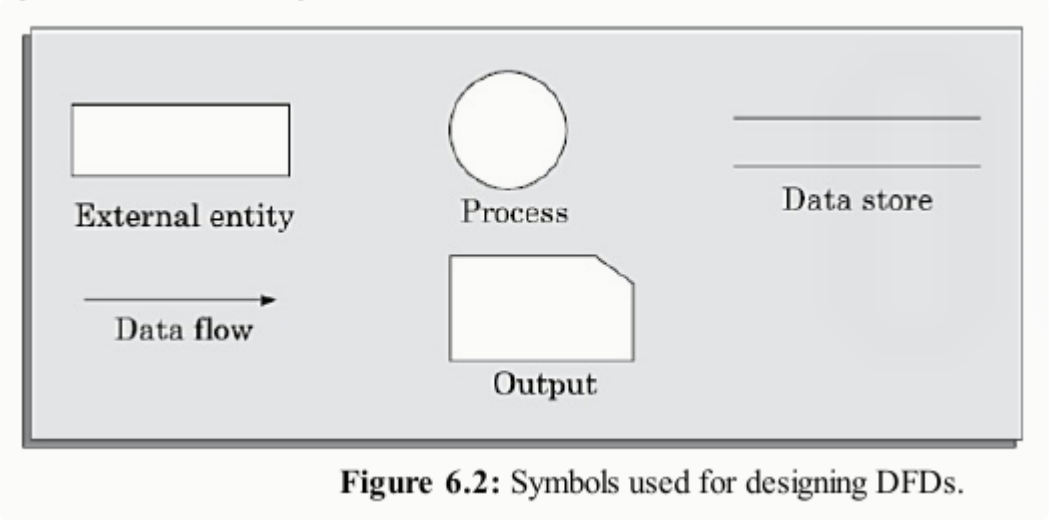


Figure 6.2: Symbols used for designing DFDs.

- **Function symbol**: A function is represented using a circle. This symbol is called a *process* or a *bubble*. Bubbles are annotated with the names of the corresponding functions.
- **External entity symbol**: The external entities are essentially those physical entities external to the software system which interact with the system by inputting data to the system or by consuming the data produced by the system. In addition to the human users, the external entity symbols can be used to represent external hardware and software such as another application software that would interact with the software being modelled. It is represented by a rectangle.
- **Data flow symbol**: A directed arc (or an arrow) is used as a data flow symbol. A data flow symbol represents the data flow occurring between two processes or between an external entity and a process in the direction of the data flow arrow. Data flow symbols are usually annotated with the corresponding data names.
- **Data store symbol**: A data store is represented using two parallel lines. It represents a logical file. That is, a data store symbol can represent either a data structure or a physical file on disk. Each data store is connected to a process by means of a data flow

symbol. The direction of the data flow arrow shows whether data is being read from or written into a data store.

- **Output symbol**: The output symbol is used when a hard copy is produced.

Synchronous operations

- If two bubbles are directly connected by a data flow arrow, then they are ***synchronous***.
- This means that they operate at the same speed.
- `validate-number` bubble can start processing only after the `read-number` bubble has supplied data to it; and the `read-number` bubble has to wait until the `validate-number` bubble has consumed its data.

Asynchronous operations

- If two bubbles are connected through a data store, as in Figure 6.3(b) then they are ***asynchronous***.
- The speed of the operations of the bubble are independent.
- The data produced by a producer bubble gets stored in the data store. It is therefore possible that the producer bubble stores several pieces of data items, even before the consumer bubble consumes any of them.

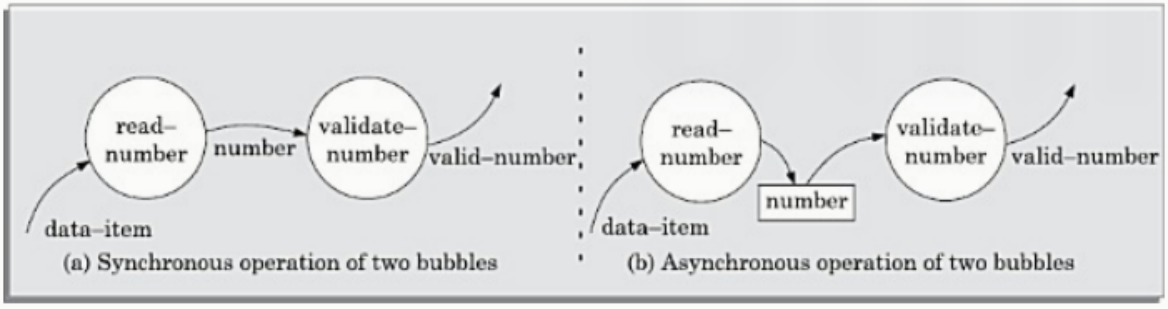


Figure 6.3: Synchronous and asynchronous data flow.

Data dictionary

- Every DFD model of a system must be accompanied by a data dictionary.
- A data dictionary lists all data items that appear in a DFD model.
- The data items listed include all data flows and the contents of all data stores appearing on all the DFDs in a DFD model.

- It lists the purpose of all data items and the definition of all composite data items in terms of their component data items.
- A data dictionary provides a standard terminology for all relevant data for use by the developers working in a project. In large projects different developers of the project have a tendency to use different terms to refer to the same data, which unnecessarily causes confusion.
- The data dictionary helps the developers to determine the definition of different data structures in terms of their component elements.
- The data dictionary helps to perform impact analysis. That is, it is possible to determine the effect of some data on various processing activities and vice versa.

Data definition

Composite data items can be defined in terms of primitive data items using the following data definition operators.

- `+` -> denotes composition of two data items, e.g. a+b represents data a and b.
- `[,,]` -> represents selection, i.e. any one of the data items listed inside the square bracket can occur
- `()` -> the contents inside the bracket represent optional data which may or may not appear. a+(b) represents either a or a+b occurs.
- `{}` -> represents iterative data definition, e.g. {name}5 represents five name data. {name}* represents zero or more instances of name data.
- `=:` -> represents equivalence, e.g. a=b+c means that a is a composite data item comprising of both b and c.
- `/* */` -> Anything appearing within is considered as comment.

Developing the DFD Model of a system

- The DFD model of a system is constructed by using a hierarchy of DFDs. The top level DFD is called the level 0 DFD or the context diagram. This is the most abstract (simplest) representation of the system (highest level).
- At each successive lower level DFDs, more and more details are gradually introduced. To develop a higher-level DFD model, processes are decomposed into their subprocesses and the data flow among these subprocesses are identified.

Context Diagram

- The context diagram is the most abstract (highest level) data flow representation of a system.
- It represents the entire system as a single bubble.
- The bubble in the context diagram is annotated with the name of the software system being developed (usually a noun). The bubbles at all other levels are annotated with verbs according to the main function performed by the bubble.
- The context diagram establishes the context in which the system operates; that is, who are the users, what data do they input to the system, and what data they received by the system.
- The various external entities with which the system interacts and the data flow occurring between the system and the external entities are represented.
- The data input to the system and the data output from the system are represented as incoming and outgoing arrows. These data flow arrows should be annotated with the corresponding data names.

Level 1 DFD

- The level 1 DFD usually contains three to seven bubbles.
- To develop the level 1 DFD, examine the high-level functional requirements in the SRS document.
- Next, examine the input data to these functions and the data output by these functions as documented in the SRS document and represent them appropriately in the diagram.
- If a system has more than seven high-level requirements some of the related requirements have to be combined and represented as a single bubble These can be split appropriately in the lower DFD levels.
- If a system has less than three high-level functional requirements, then some of the high-level requirements need to be split into their subfunctions

Decomposition

- Decomposition of a bubble is also known as ***factoring*** or ***exploding*** a bubble.
- Each bubble at any level of DFD is usually decomposed to anything three to seven bubbles.

- Decomposition of a bubble should be carried on until a level is reached at which the function of the bubble can be described using a simple algorithm.

Numbering of bubbles

- These numbers help in uniquely identifying any bubble in the DFD.
- The bubble at the context level is usually assigned the number 0
- Bubbles at level 1 are numbered, 0.1, 0.2, 0.3, etc.
- When a bubble numbered x is decomposed, its children bubble are numbered x.1, x.2, x.3, etc. Such that by looking at the number of a bubble we can unambiguously determine its level, its ancestors, and its successors.

Balancing DFDs

- The data that flow into or out of a bubble must match the data flow at the next level of DFD. This is known as ***balancing a DFD.***

How far to decompose?

- A bubble should not be decomposed any further once a bubble is found to represent a simple set of instructions.
- For simple problems, decomposition up to level 1 should suffice.
- However, large industry standard problems may need decomposition up to level 3 or level 4.

Things to remember while making DFDs

- All external entities interacting with the system should be represented only in the context diagram. The external entities should not appear in the DFDs at any other level.
- Only three to seven bubbles per diagram should be allowed.

Shortcomings of a DFD model

- Imprecise DFDs leave ample scope to be imprecise. For example, the `find-book-position` bubble may not convey anything regarding what happens when the required book is missing.
- The order in which inputs are consumed and outputs are produced by a bubble is not specified. A DFD model does not specify the order in which the different bubbles are executed. Representation of such aspects is very important for modelling real-time systems.

- Decomposition: The method of carrying out decomposition to arrive at the successive levels and the ultimate level to which decomposition is carried out are highly subjective and depend on the choice and judgment of the analyst.

Structured Design

- During structured design, the DFD model is transformed into a ***structure chart***.
- During structured design, all functions identified during structured analysis are mapped to a module structure. This module structure is also called the ***high-level design*** or the ***software architecture*** for the given problem, represented using structure chart.
- The various modules making up the system, the module dependency (i.e. which module calls which other modules), and the parameters that are passed among the different modules.
- It is normally followed by a detailed design stage. During the detailed design stage, the algorithms and data structures for the individual modules are designed. The detailed design can directly be implemented as a working system using a conventional programming language.

Making of structure chart

Symbols

- **Rectangular boxes**: A rectangular box represents a module. Usually, every rectangular box is annotated with the name of the module.
- **Module invocation arrows**: An arrow connecting two modules implies that during program execution control is passed from one module to the other in the direction of the connecting arrow. The frequency and the order in which the modules are invoked is not relevant to the structure chart.
- **Data flow arrows**: These are small arrows appearing alongside the module invocation arrows. The data flow arrows are annotated with the corresponding data name. Data flow arrows represent the named data passes from one module to the other in the direction of the arrow.
- **Library modules**: A library module is usually represented by a rectangle with double edges. Libraries comprise the frequently called

modules. Usually, when a module is invoked by many other modules, it is made into a library module.

- **Selection**: The diamond symbol represents the fact that one module of several modules connected with the diamond symbol is invoked depending on the outcome of the condition attached with the diamond symbol.
- **Repetition**: A loop around the control flow arrows denotes that the respective modules are invoked repeatedly.

Rules

- There should be one and only one module at the top, called the ***root***.
- There should be at most one module invocation arrow between any two modules, i.e., if module A invokes module B, module B cannot invoke module A.

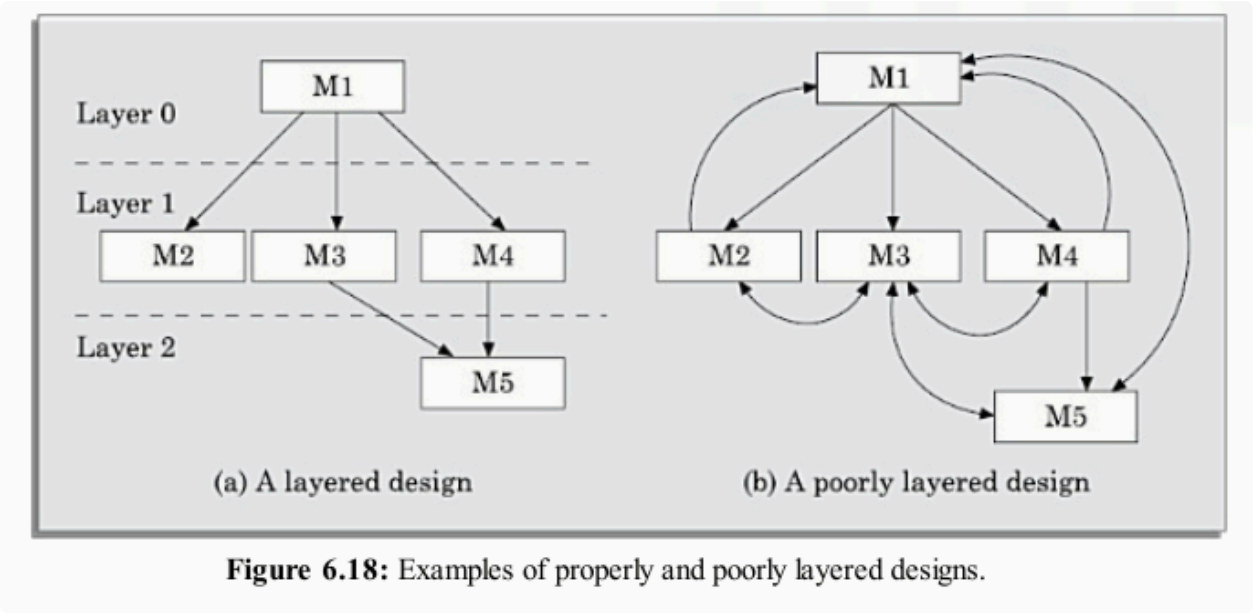


Figure 6.18: Examples of properly and poorly layered designs.

Flow chart vs Structure chart

A structure chart differs from a flow chart in three principal ways:

- It is usually difficult to identify the different modules of a program from its flow chart representation.
- Data interchange among different modules is not represented in a flow chart.
- Sequential ordering of tasks that is inherent to a flow chart is suppressed in a structure chart.

Self-study: Transformation of a DFD model into structure chart.

Detailed Design

- During detailed design the pseudo code description of the processing and the different data structures are designed for the different modules of the structure chart. Usually described in the form of module specification (MSPEC).

MSPEC

- MSPEC is usually written using structured English.
- The MSPEC for the non-leaf modules describe the different conditions under which the responsibilities are delegated to the lower-level modules.
- The MSPEC for the leaf-level modules should describe in algorithmic form how the primitive processing steps are carried out.
- To develop the MSPEC of a module, it is usually necessary to refer to the DFD model and the SRS document to determine the functionality of the module.

Design Review

- After a design is complete, the design is required to be reviewed.
- The review team usually consists of members with design, implementation, testing, and maintenance perspectives, who may or may not be the members of the development team.
- The review team checks the design documents especially for the following aspects:
  - **Traceability**: Whether each bubble of the DFD can be traced to some module in the structure chart and vice versa. Also, whether each functional requirement in the SRS document can be traced to some bubble in the DFD model and vice versa.
  - **Correctness**: Whether all the algorithms and data structures of the detailed design are correct.
  - **Maintainability**: Whether the design can be easily maintained in future.
  - **Implementation**: Whether the design can be easily and efficiently be implemented.